CHAPTER 03

Spring Framework

IoC · AOP · MVC · Security · Data · Boot 3

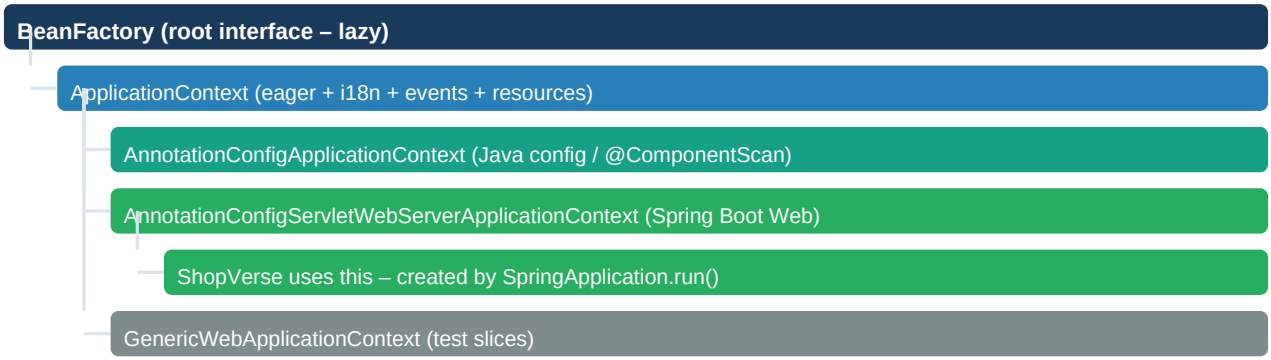
Chapter Overview

Chapter 03 covers the Spring Framework as used throughout ShopVerse: IoC container internals, bean lifecycle, AOP, Spring MVC request processing, Spring Security filter chain, Spring Data, Spring Events, auto-configuration, actuator, and Spring Boot 3 features.

Sub-Chapter	Topic	Key ShopVerse Class(es)
03-01	IoC Container & Dependency Injection	ApplicationContext, @Bean, @Component, @Autowired
03-02	Bean Lifecycle	BeanPostProcessor, @PostConstruct, @PreDestroy, SmartLifecy
03-03	AOP – Aspect-Oriented Programming	@Transactional, @Cacheable, @Async, SecurityAspect, AuditAs
03-04	Spring MVC Request Pipeline	DispatcherServlet, HandlerMapping, @RestController, Exceptionl
03-05	Spring Security	SecurityFilterChain, JwtAuthFilter, UserDetailsService, @Secured
03-06	Spring Data JPA	JpaRepository, @Query, Specification, Projections, Auditing
03-07	Spring Events	ApplicationEventPublisher, @EventListener, @TransactionalEven
03-08	Auto-configuration & Spring Boot 3	@SpringBootApplication, @ConditionalOnProperty, Virtual Threa
03-09	Spring Actuator & Observability	/actuator, Micrometer, custom endpoints

03-01 IoC Container & Dependency Injection

ApplicationContext Hierarchy



Injection Styles Comparison

Style	Syntax	Pros	Cons	Verdict
Constructor	@Autowired on constructor (optional since Spring 4.3+)	Mandatory deps clear; immutable; testable	Verbose with many deps	■ PREFERRED in ShopVer
Field	@Autowired on field	Compact	Hides dependencies; not testable without Spring; cannot b	■ Avoid
Setter	@Autowired on setter	Optional deps clear	Mutable; harder to track required deps	■ Use for optional deps only

```
// Constructor injection (ShopVerse pattern)
@Service public class OrderService {
private final OrderRepository repo;
private final InventoryService inventory;
private final PaymentGateway payment;
// @Autowired not needed since Java 4.3 single-constructor rule
public OrderService(OrderRepository r, InventoryService i, PaymentGateway p) {
this.repo = r; this.inventory = i; this.payment = p;
}
}
```

Scopes

Scope	Annotation	One Instance Per	ShopVerse Usage
singleton	(default)	ApplicationContext	All @Service, @Repository, @Component
prototype	@Scope("prototype")	inject call	Stateful objects (avoid in ShopVerse)
request	@RequestScope	HTTP request	JWT claims holder, per-request locale
session	@SessionScope	HTTP session	Shopping cart (alternative to Redis)
application	@ApplicationScope	ServletContext	Rarely used; prefer singleton

@Qualifier & @Primary for Disambiguation

```
// Multiple PaymentGateway implementations
@Component("stripe") public class StripeAdapter implements PaymentGateway {...}
@Component("paypal") public class PayPalAdapter implements PaymentGateway {...}
@Primary @Component public class StripeAdapter implements PaymentGateway {...}
// Injection
public OrderService(@Qualifier("stripe") PaymentGateway gateway) {...}
// Or inject all:
public OrderService(List gateways) {...} // all implementations
```

03-02 **Bean Lifecycle**

Full Bean Lifecycle Sequence

1. Instantiation – constructor called
2. Dependency Injection – @Autowired fields/setters populated
3. BeanNameAware.setBeanName()
4. BeanFactoryAware.setBeanFactory()
5. ApplicationContextAware.setApplicationContext()
6. BeanPostProcessor.postProcessBeforeInitialization() – all registered BPP
7. @PostConstruct method / InitializingBean.afterPropertiesSet()
8. BeanPostProcessor.postProcessAfterInitialization() – AOP proxy created h
9. Bean is READY – serving requests
10. ApplicationContext.close() called
11. @PreDestroy / DisposableBean.destroy()

```
// ShopVerse ApplicationStartupValidator
@Component public class ApplicationStartupValidator implements SmartLifecycle {
private boolean running = false;
@Override public void start() {
validateDatabaseConnection();
validateRedisConnection();
validateKafkaConnection();
running = true;
log.info("All infrastructure dependencies verified");
}
@Override public boolean isAutoStartup() { return true; }
@Override public int getPhase() { return Integer.MAX_VALUE; } // last to start
}
// Simple lifecycle hooks
@PostConstruct public void init() { log.info("OrderService ready"); }
@PreDestroy public void shutdown() { executor.shutdownNow(); }
```

BeanPostProcessor Use Cases

BPP	Purpose	Spring Example
AutowiredAnnotationBeanPostProcessor	Processes @Autowired	Core Spring – always active
CommonAnnotationBeanPostProcessor	Processes @PostConstruct, @PreDestroy	Core Spring – always active
PersistenceAnnotationBeanPostProcessor	Processes @PersistenceContext	Spring Data JPA
AsyncAnnotationBeanPostProcessor	Wraps @Async methods in proxy	@EnableAsync
Custom BPP	Modify any bean before/after init	Metrics, validation, decoration

AOP Terminology

Term	Definition	ShopVerse Example
Aspect	Cross-cutting concern module	AuditAspect, SecurityAspect, PerformanceAspect
Advice	Code that runs at join point	@Before, @After, @Around, @AfterReturning, @AfterThrowing
Join Point	Point during execution (method call)	Any method call in Spring proxied beans
Pointcut	Expression selecting join points	@Pointcut("execution(* com.shopverse.service.*.*(..))")
Weaving	Applying aspects to targets	Spring uses runtime proxy weaving (not compile/load time)
Proxy	Wrapped object with advice injected	JDK Dynamic Proxy (interface) or CGLIB (class)

```
// AuditAspect – logs all service method calls
@Aspect @Component public class AuditAspect {
@Pointcut("execution(* com.shopverse.service.*Service.*(..))")
private void serviceMethods() {}
@Around("serviceMethods()")
public Object audit(ProceedingJoinPoint pjp) throws Throwable {
String method = pjp.getSignature().toShortString();
Instant start = Instant.now();
try {
Object result = pjp.proceed();
log.info("OK {} {}ms", method, Duration.between(start, Instant.now()).toMillis());
return result;
} catch (Exception e) {
log.error("FAIL {} {}", method, e.getMessage()); throw e;
}
}
}
```

@Transactional Deep Dive

Attribute	Default	ShopVerse Override	Notes
propagation	REQUIRED	REQUIRES_NEW for audit	REQUIRED: join existing. REQUIRES_NEW: suspend & start
isolation	DEFAULT (DB default = RC)	REPEATABLE_READ for flash sale	Prevents non-repeatable reads in concurrent flash sale
readOnly	false	true on all read methods	Hints Hibernate to skip dirty checking; may use read replica
timeout	–1 (no limit)	30 (seconds)	Throws TransactionTimedOutException after 30s
rollbackFor	RuntimeException + Error	Exception.class	Include checked exceptions in rollback
noRollbackFor	(none)	OptimisticLockException	Retry externally; don't rollback whole tx

```
@Transactional(readOnly=true, timeout=10)
public OrderResponse getOrder(long id) { return mapper.toDto(repo.findById(id).orElseThrow());
}
@Transactional(isolation=REPEATABLE_READ, timeout=30, rollbackFor=Exception.class)
public OrderResponse placeOrder(PlaceOrderRequest req) { ... }
```

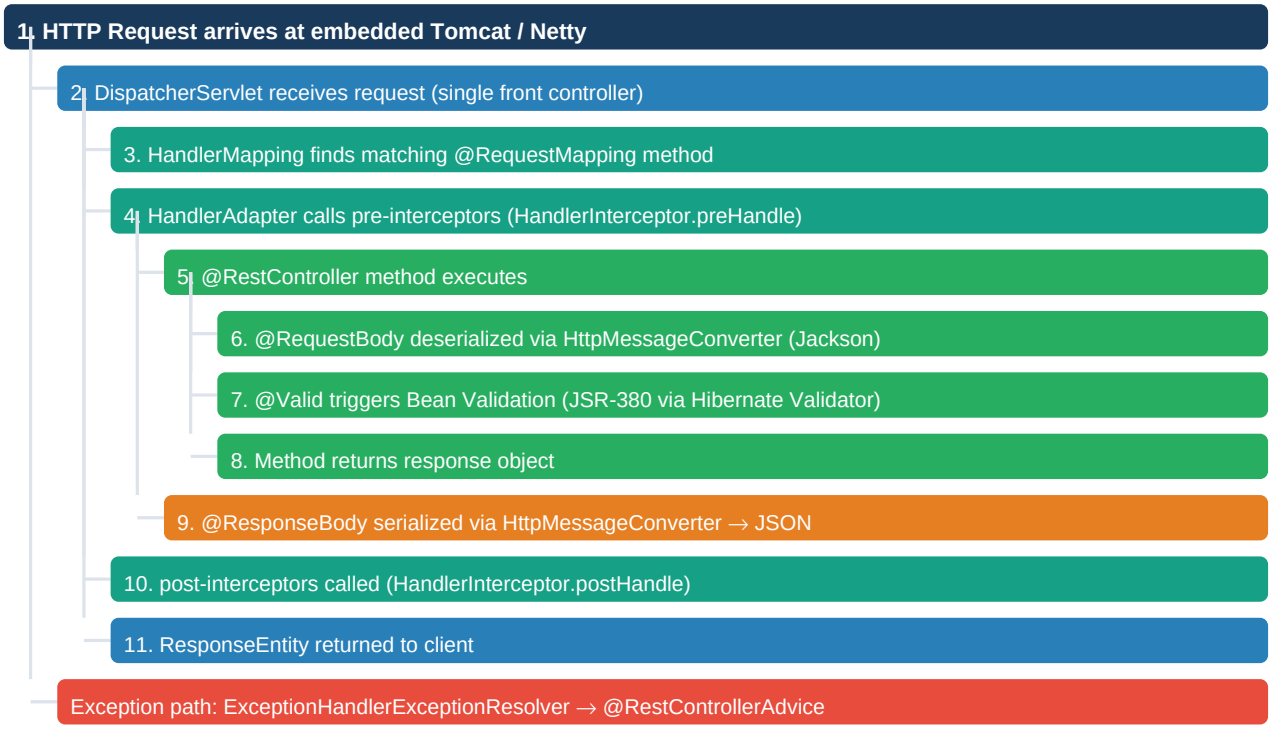
@Cacheable / @CachePut / @CacheEvict

Annotation	Behaviour	ShopVerse Usage
@Cacheable(key="#id")	Return cached value if present; else call method and cache result	ProductService.findById – cache product lookups
@CachePut(key="#result.id")	Always call method AND update cache	ProductService.update – keep cache current
@CacheEvict(key="#id")	Remove entry on call	ProductService.delete / status change

Annotation	Behaviour	ShopVerse Usage
@Caching({evict,put})	Combine multiple cache operations	OrderService.placeOrder – evict cart, put order

03-04 **Spring MVC Request Pipeline**

Request Flow



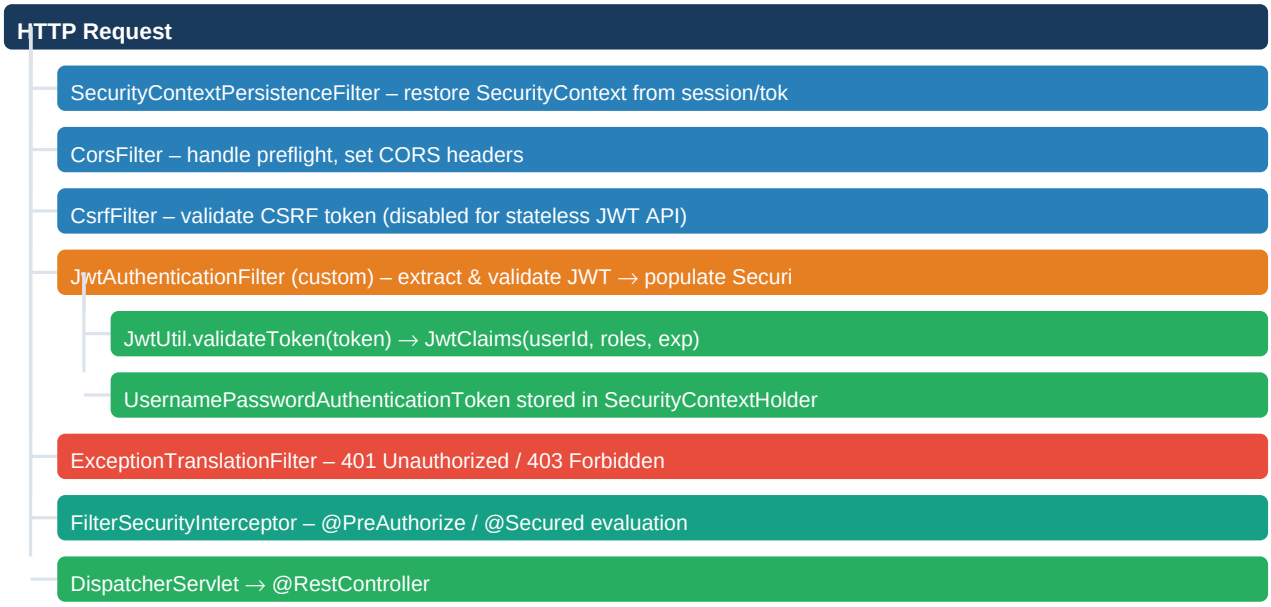

```
// OrderController – ShopVerse pattern
@RestController @RequestMapping("/api/v1/orders")
@RequiredArgsConstructor @Validated
public class OrderController {
private final OrderFacade orderFacade;
@PostMapping
@PreAuthorize("hasRole('CUSTOMER')")
public ResponseEntity> placeOrder(
@RequestBody @Valid PlaceOrderRequest req,
@AuthenticationPrincipal UserDetails user) {
OrderResponse resp = orderFacade.placeOrder(req);
return ResponseEntity.status(CREATED).body(ApiResponse.success(resp));
}
@GetMapping("/{id}")
public ResponseEntity> get(@PathVariable long id) {
return ResponseEntity.ok(ApiResponse.success(orderFacade.getOrder(id)));
}
}
```

Bean Validation Annotations Used in ShopVerse

Annotation	Applies To	ShopVerse Field
@NotNull	Any	order.customerId, payment.amount
@NotBlank	String	product.name, customer.email
@Email	String	Customer.email
@Min / @Max	Number	CartItem.quantity (min=1, max=100)
@Size(min,max)	String/Collection	Product.description (max=2000)

Annotation	Applies To	ShopVerse Field
@Positive	Number	Product.price, Order.total
@Pattern(regexp)	String	Phone number, postal code
@Valid (cascade)	Object field	Order.items validates each OrderItem

Security Filter Chain Order




```
// JwtAuthenticationFilter
@Component public class JwtAuthenticationFilter extends OncePerRequestFilter {
@Override protected void doFilterInternal(HttpServletRequest req,
HttpServletResponse res, FilterChain chain) throws IOException, ServletException {
String header = req.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
String token = header.substring(7);
if (jwtUtil.validateToken(token)) {
JwtClaims claims = jwtUtil.extractClaims(token);
var auth = new UsernamePasswordAuthenticationToken(
claims.userId(), null,
claims.roles().stream().map(SimpleGrantedAuthority::new).toList());
SecurityContextHolder.getContext().setAuthentication(auth);
}
}
chain.doFilter(req, res);
}
}
```

SecurityFilterChain Configuration

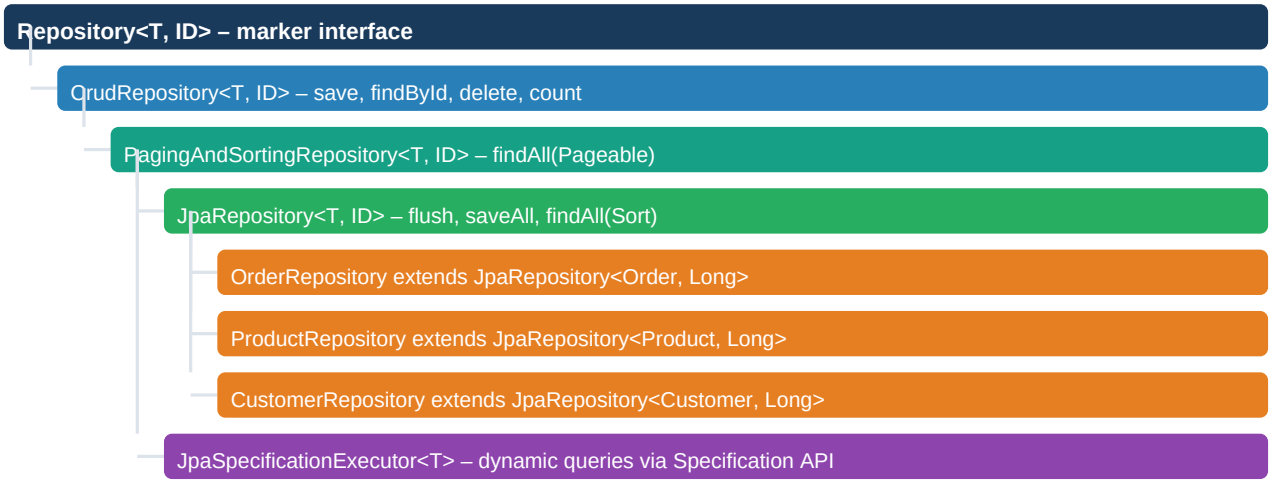
```
@Configuration @EnableMethodSecurity
public class SecurityConfig {
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
return http
.csrf(csrf -> csrf.disable()) // stateless JWT API
.cors(cors -> cors.configurationSource(corsConfigurationSource()))
.sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
.authorizeHttpRequests(a -> a
.requestMatchers("/api/v1/auth/**").permitAll()
.requestMatchers("/actuator/health").permitAll()
.requestMatchers(HttpMethod.GET, "/api/v1/products/**").permitAll()
.anyRequest().authenticated())
.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
.exceptionHandling(e -> e
.authenticationEntryPoint((req,res,ex) -> res.sendError(401))
```

```
.accessDeniedHandler((req,res,ex) -> res.sendError(403)))
.build();
}
}
```

Roles & Authorities

Role	Endpoints Permitted	ShopVerse Annotation
ANONYMOUS	GET /products, GET /categories, POST /auth/**	permitAll()
CUSTOMER	POST /orders, GET /orders/my, PUT /cart, GET /products	@PreAuthorize("hasRole('CUSTOMER')")
ADMIN	DELETE /products, PUT /products, GET /orders (all)	@PreAuthorize("hasRole('ADMIN')")
SELLER	POST /products, PUT /products/{id}, GET /orders (own)	@PreAuthorize("hasRole('SELLER')")

Repository Hierarchy




```
public interface OrderRepository extends JpaRepository,
JpaSpecificationExecutor {
// Derived query method
List findByCustomerIdAndStatusOrderByCreatedAtDesc(long customerId, OrderStatus status);
// JPQL @Query with named params
@Query("SELECT o FROM Order o WHERE o.customerId = :cid AND o.createdAt > :since")
List findRecentOrders(@Param("cid") long customerId, @Param("since") Instant since);
// Native SQL for complex queries
@Query(value="SELECT * FROM orders WHERE total_amount > :min ORDER BY created_at DESC LIMIT
:lim",
nativeQuery=true)
List findHighValueOrders(@Param("min") BigDecimal min, @Param("lim") int lim);
// Projection – only select needed columns
List findByCustomerId(long customerId); // interface projection
}
```

Specification API for Dynamic Search

```
// ProductSearchCriteria → Specification
public class ProductSpecification {
public static Specification withCategory(long catId) {
return (root, q, cb) -> cb.equal(root.get("categoryId"), catId); }
public static Specification withMinPrice(BigDecimal min) {
return (root, q, cb) -> cb.greaterThanOrEqualTo(root.get("price"), min); }
public static Specification withNameContaining(String kw) {
return (root, q, cb) -> cb.like(cb.lower(root.get("name")), "%"+kw.toLowerCase()+"%"); }
}
// Chaining specifications
Specification spec = Specification.where(withCategory(catId))
.and(withMinPrice(minPrice))
.and(withNameContaining(keyword));
Page results = productRepo.findAll(spec, PageRequest.of(page, size, Sort.by("name")));
```

Auditing with @EnableJpaAuditing

```
@EntityListeners(AuditingEntityListener.class) @MappedSuperclass
public abstract class BaseEntity {
@CreatedDate @Column(updatable=false) private Instant createdAt;
@LastModifiedDate private Instant updatedAt;
@CreatedBy @Column(updatable=false) private String createdBy;
@LastModifiedBy private String updatedBy;
```

```
@Version private int version; // optimistic lock
}
```

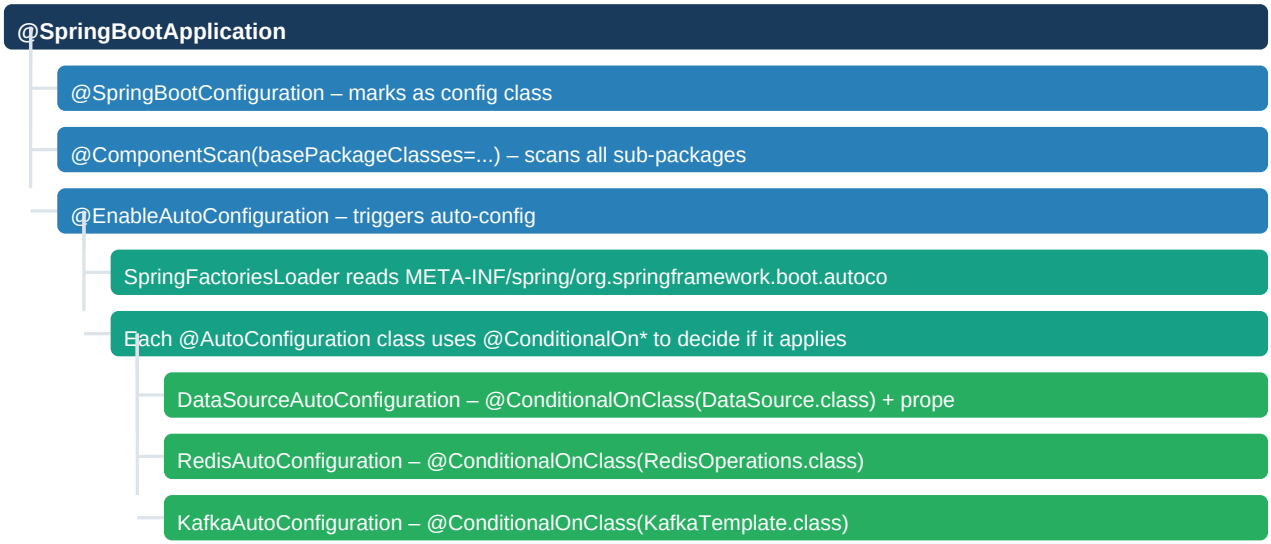
@EventListener vs @TransactionalEventListener

Annotation	Phase	Transaction	Use Case
@EventListener	Immediately on publish	Shares publisher's transaction	Sync in-transaction side effects (inventory deduction)
@EventListener @Async	Async thread	New or no transaction	Notifications, analytics – non-critical
@TransactionalEventListener (phase = AFTER_COMMIT)	After tx commits	Outside original tx	Send email only if order actually saved
@TransactionalEventListener (phase = AFTER_ROLLBACK)	After tx rolls back	Outside original tx	Compensating action on failure

```
// OrderPlacedEvent – record DTO
public record OrderPlacedEvent(long orderId, long customerId, BigDecimal total, Instant
placedAt) {}
// Publisher
@Service public class OrderService {
private final ApplicationEventPublisher pub;
@Transactional public Order create(PlaceOrderRequest req) {
Order o = repo.save(mapToEntity(req));
pub.publishEvent(new OrderPlacedEvent(o.getId(), o.getCustomerId(), o.getTotal(),
Instant.now()));
return o;
}
}
// Send email ONLY after DB commit (AFTER_COMMIT guarantees order is durable)
@Component public class OrderConfirmationListener {
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Async
public void sendConfirmation(OrderPlacedEvent event) {
notificationService.sendOrderConfirmationEmail(event.orderId(), event.customerId());
}
}
```

03-08 **Auto-configuration & Spring Boot 3**

@SpringBootApplication Composition



@ConditionalOn* Annotations

Annotation	Condition	ShopVerse Usage
@ConditionalOnProperty	Property present with value	@ConditionalOnProperty("shopverse.cache.enabled") on CacheConfig
@ConditionalOnClass	Class on classpath	Auto-configure Redis only if lettuce jar present
@ConditionalOnMissingBean	No bean of type registered	Auto-configure default impl only if no custom bean
@ConditionalOnExpression	SpEL evaluates true	@ConditionalOnExpression("${env}=='prod'")
@ConditionalOnWebApplication	Web context	Only apply web-specific beans in servlet apps
@Profile	Active Spring profile matches	@Profile("prod") on ProdDataSourceConfig

```
// Custom conditional configuration
@Configuration
@ConditionalOnProperty(name="shopverse.feature.flashsale.enabled", havingValue="true")
public class FlashSaleAutoConfiguration {
@Bean public FlashSaleService flashSaleService(RedisTemplate redis,
ProductRepository products) {
return new FlashSaleService(redis, products);
}
}
```

Spring Boot 3 Key Features Used

Feature	Config / Class	Description
Virtual Threads (Loom)	spring.threads.virtual.enabled=true	All HTTP, @Async, scheduler threads use virtual threads
Problem Details (RFC 7807)	spring.mvc.problemdetails.enabled=true	Structured error responses: type, title, status, detail, instance
Micrometer Observation API	@Observed on @Service	Auto-create spans + metrics per method
Native Image (GraalVM)	mvn -Pnative spring-boot:build-image	AOT compilation; ~10x faster startup; lower memory
Jakarta EE 10	jakarta.* packages	Spring Boot 3 migrated from javax.* to jakarta.*
HTTP Interface Clients	@HttpExchange	Declarative REST clients like Feign (no Feign dep needed)

Key Actuator Endpoints

Endpoint	Default Exposure	ShopVerse Usage
/actuator/health	Web + JMX	Kubernetes liveness (live) + readiness (ready) probes
/actuator/info	JMX only (expose in config)	Git commit, build version, Java version metadata
/actuator/metrics	JMX only (expose)	Micrometer metrics: JVM, HTTP, custom business metrics
/actuator/prometheus	Not exposed by default	Prometheus scrape endpoint for Grafana dashboards
/actuator/loggers	JMX only	Runtime log level changes without restart
/actuator/env	JMX only	View resolved property sources and values
/actuator/flyway	JMX only	Show applied and pending DB migrations
/actuator/jvm-metrics (custom)	Web	Custom endpoint showing heap usage + GC pause ms

```
# application.yml – expose actuator endpoints
management:
endpoints.web.exposure.include: health,info,metrics,prometheus,loggers
endpoint.health.show-details: when-authorized
health.probes.enabled: true # enables /health/liveness + /health/readiness
metrics.tags.application: shopverse # common tag on all metrics
tracing.sampling.probability: 1.0 # 100% trace sampling in dev
# Custom health indicator
@Component public class KafkaHealthIndicator implements HealthIndicator {
public Health health() {
try { admin.listTopics().names().get(5, SECONDS);
return Health.up().withDetail("brokers", kafkaProps.getBootstrapServers()).build(); }
catch (Exception e) { return Health.down().withException(e).build(); }
}
}
```

Custom Metrics with Micrometer

```
@Service public class OrderService {
private final Counter ordersPlaced;
private final Timer orderProcessingTime;
public OrderService(MeterRegistry registry) {
ordersPlaced = Counter.builder("shopverse.orders.placed")
.description("Total orders placed").register(registry);
orderProcessingTime = Timer.builder("shopverse.orders.processing.time")
.description("Order processing duration").register(registry);
}
public OrderResponse placeOrder(PlaceOrderRequest req) {
return orderProcessingTime.record(() -> {
Order o = createOrder(req);
ordersPlaced.increment();
return mapper.toDto(o);
});
}
}
```

BP Best Practices & Anti-Patterns

Do

- ✓ Use constructor injection for mandatory dependencies; setter for optional.
- ✓ Annotate all read-only service methods with @Transactional(readOnly=true).
- ✓ Use @TransactionalEventListener(AFTER_COMMIT) for side effects that depend on data being committed.
- ✓ Expose /actuator/health probes separately for liveness and readiness (Kubernetes).
- ✓ Pre-size ArrayBlockingQueue and thread pools; set max to prevent resource exhaustion.
- ✓ Use @Valid on @RequestBody to trigger Bean Validation automatically.
- ✓ Apply @EnableMethodSecurity with @PreAuthorize at controller level for fine-grained auth.
- ✓ Set spring.threads.virtual.enabled=true in Spring Boot 3.2+ for IO-bound workloads.
- ✓ Use Specification API instead of multiple findByX methods for complex dynamic queries.

Anti-Patterns

- ■ Field injection (@Autowired on fields) – hides dependencies; untestable without Spring context.
- ■ Calling @Transactional methods from within the same class (self-invocation bypasses proxy).
- ■ Catching and swallowing TransactionSystemException in service – leaves resources in unknown state.
- ■ Putting business logic in @Entity classes – creates circular dependencies with repositories.
- ■ Using @Transactional on controller methods – transaction should wrap service, not HTTP layer.
- ■ Exposing all actuator endpoints without security – leaks internals to unauthenticated callers.
- ■ Using @Async without configuring a custom thread pool – defaults to SimpleAsyncTaskExecutor (no pooling).
- ■ LazyInitializationException – accessing lazy-loaded associations outside a transaction boundary.

QR Quick Reference

Sub-Chapter	Key Concept	One-Liner
03-01 IoC	Constructor injection	Preferred: mandatory deps clear; final fields; mockable
03-02 Lifecycle	@PostConstruct	Runs after DI; use for init work (validate, cache warm)
03-03 AOP	@Around advice	Full control: before, after, catch, return value
03-04 MVC	DispatcherServlet	Single front controller; routes via HandlerMapping
03-05 Security	JwtAuthFilter	OncePerRequestFilter; validates JWT; populates SecurityContext
03-06 Data	Specification API	Dynamic queries via Specification.where().and()
03-07 Events	AFTER_COMMIT	TransactionalEventListener: side effects only after successful commit
03-08 Boot 3	Virtual Threads	spring.threads.virtual.enabled=true – millions of lightweight threads
03-09 Actuator	/health/readiness	Kubernetes readiness probe; returns DOWN until deps healthy

Interview Quick-Check

Question	Concise Answer
@Component vs @Bean?	@Component: class-level auto-scan. @Bean: method-level in @Configuration; use when you don't
BeanPostProcessor vs @PostConstruct?	BPP: intercepts ALL beans; used by framework (AOP proxy creation). @PostConstruct: single bea
@Transactional self-invocation?	Spring proxy not invoked on this.method(). Solution: inject self, use AspectJ, or restructure.
readOnly=true effect?	Hints Hibernate to skip dirty checking; may route to read replica; no dirty-check overhead.
@EventListener vs Kafka?	@EventListener: in-process, same JVM. Kafka: cross-service, durable, replay-capable.
Filter vs Interceptor?	Filter: Servlet-level, before DispatcherServlet. Interceptor: Spring-level, after routing, can access ha